

Contents

LLM End-to-End Notes — Scratch se Advance tak	1
0. Big Picture — 3 Stages of Building an LLM	1
1. STAGE 1 — Pre-training (Knowledge Building)	2
1.1 Data Collection	2
1.2 Data Processing Pipeline (raw internet → clean training data)	2
1.3 Tokenization — Text ko Model-Readable Banane ka Process	2
1.4 Neural Network Training	3
2. Inference — Model se Output Kaise Nikalta Hai	3
3. Base Model ka Real Behavior — Hallucination aur In-Context Learning	4
3.1 Hallucination kya hai	4
3.2 Few-shot Prompting (Base model ko useful banane ka trick)	4
4. STAGE 2 — Post-Training: Supervised Fine-Tuning (SFT) → Assistant Banana	4
4.1 Conversation Tokenization — Special Tokens	5
5. LLM Psychology — Model ke “Dimaag” ko Samajhna	5
6. Limitations — Model Kyun Kuch Cheezon Mein Weak Hai	5
6.1 Counting & Spelling problem	5
6.2 Per-token computation limit	5
7. Mitigating Hallucinations (Model ko Zyada Truthful Banana)	6
8. STAGE 3 — Reinforcement Learning (RL): Reasoning Sikhaana	6
8.1 RL kya hai	6
8.2 Verifiable Domains	7
8.3 AlphaGo Analogy (bahut powerful example)	7
8.4 Unverifiable Domains ka Challenge — RLHF	7
8.5 RLHF ki Limitation	7
9. Quick Comparison — GPT-2 vs Llama 3 (Scale ka Evolution)	8
10. Thinking Models — DeepSeek, o1/o3, Gemini Thinking	8
11. Practical Guide — LLM ko Effectively Use Kaise Karein	8
12. Future of LLMs (Karpathy ki Predictions)	9
13. Staying Updated (Resources jo Karpathy suggest karte hain)	9
14. One-Page Cheat Sheet (Revision ke liye)	9
Glossary (Key Terms — Hinglish mein)	10

LLM End-to-End Notes — Scratch se Advance tak

Reference video: Andrej Karpathy — “Deep Dive into LLMs like ChatGPT” (youtube.com/watch?v=7xTGNNLF)

Purpose: Ek baar padh ke LLM ka poora pipeline samajh aa jaye — kya hai, kaise banta hai, kyun aisa behave karta hai, aur use kaise karein.

Karpathy ki line jo pura video sum up karti hai: *“It is obviously magical and amazing in some respects. It’s really good at some things, not very good at other things, and there’s also a lot of sharp edges to be aware of.”* Matlab — LLM ek powerful tool hai, god nahi. Blind trust mat karo, samajh ke use karo.

0. Big Picture — 3 Stages of Building an LLM

Har modern LLM (ChatGPT, Llama, Gemini, Claude) teen major stages se guzarta hai:

Stage	Kaam kya hota hai	Output
1. Pre-training	Pura internet padhao, next-word predict karna sikhao	Base model (raw knowledge, but not an assistant)
2. Post-training → SFT	Human-written ideal conversations pe fine-tune karo	Assistant model (chat karna aata hai)
3. Post-training → RL	Trial-and-error se practice karo, best solutions khud discover karo	Reasoning/thinking model (jaise o1, DeepSeek-R1)

Simple analogy (Hinglish): Pre-training = school mein saara syllabus ratna (raw knowledge). SFT = teacher ke saath practice karna ki answer kaise likhna hai (format/style). RL = khud practice papers solve karke apni strategy improve karna (jaise ek student jo trial-error se apna best method dhoondta hai).

1. STAGE 1 — Pre-training (Knowledge Building)

1.1 Data Collection

- Sabse pehle **internet se text data** download hota hai — massive scale pe.
- Example: **Common Crawl** — 2007 se internet crawl kar raha hai, **2.7 billion+ web pages** indexed.
- Raw internet data bahut messy hai — isliye heavy filtering hoti hai.

1.2 Data Processing Pipeline (raw internet → clean training data)

1. **URL filtering** — spam, adult, low-quality sites hata do.
2. **Text extraction** — HTML se sirf actual readable text nikalo (ads, nav-bars, boilerplate hatao).
3. **Language filtering** — kis language ka content chahiye woh rakho (e.g., English-heavy).
4. **PII removal** — personal info (addresses, SSN, phone numbers) hata do privacy ke liye.
5. **Deduplication + quality filtering** — repeated/low quality docs hatao.

Karpathy: *“We want large diversity of high-quality documents, and we want many, many of them.”*

- Example dataset: Hugging Face ka **FineWeb** — ek curated, high-quality version of Common Crawl.
- Final result: kuch **trillion words / tokens** ka clean text corpus (jaise ek text file jitna bada ki agar print karo to imagine hi nahi kar sakte — but sirf text hai, ek dimension mein).

Mera addition (extra point): Data quality > Data quantity. Garbage in, garbage out — agar training data biased ya low quality hai to model bhi wahi seekhega. Isi liye har company (OpenAI, Meta, Anthropic) apna khud ka data pipeline banati hai — ye ek competitive advantage hai.

1.3 Tokenization — Text ko Model-Readable Banane ka Process

- Neural network raw text (characters) directly nahi samajh sakta — usse numbers chahiye.

- **Tokenization** = text ko chhote-chhote pieces (tokens) mein todna, phir har piece ko ek number (ID) assign karna.
- Method used: **Byte Pair Encoding (BPE)** — jo frequently-occurring byte/character sequences ko combine karke ek single token bana deta hai (isse sequence length chhoti ho jaati hai, computation efficient hota hai).

Example (simple):

Text: "ChatGPT is helpful"

Tokens (approx): ["Chat", "G", "PT", " is", " helpful"]

Token IDs: [8206, 38, 2898, 318, 7613]

Real models (GPT-4) ~100,000+ unique tokens ka vocabulary use karte hain.

Karpathy: *"The process of converting from raw text into these symbols... is called tokenization."*

Kyun important hai samajhna: Model **characters nahi dekhta, sirf tokens dekhta hai**. Isi wajah se LLMs ko counting, spelling jaise simple tasks mein dikkat aati hai (isko hum aage detail mein dekhenge — Section 6).

1.4 Neural Network Training

- Architecture: **Transformer** (2017 paper "Attention is All You Need" se aaya) — sequences ko efficiently process karne ke liye designed.
- Network mein **billions of parameters (weights)** hote hain — training shuru hone se pehle ye **random** hote hain.
- **Training objective: next token prediction.** Model ko diya jaata hai tokens ka ek sequence, aur usse predict karna hota hai — "agla token kya aayega?"
- Prediction galat hone par, **backpropagation** se weights thoda adjust kiye jaate hain taaki agli baar prediction better ho.
- Ye process **trillions of tokens** par lakhon baar repeat hota hai (bahut GPUs, bahut electricity, weeks-months ka time).

Karpathy: *"We're trying to basically predict the token that comes next in the sequence."*

Simple analogy: Socho ek bachha jo lakhon kitabein padh raha hai aur har sentence ke beech mein guess kar raha hai "next word kya hoga" — jab galat guess kare to correct kiya jaata hai. Lakhon repetitions ke baad woh language ka pattern, facts, grammar, reasoning style sab implicitly seekh leta hai.

Result: BASE MODEL - Ye ek "internet document simulator" hai — jo bhi text diya jaaye, uska statistically-likely continuation generate karta hai. - **Ye assistant NAHI hai abhi** — ye sirf internet text ko continue karna jaanta hai.

2. Inference — Model se Output Kaise Nikalta Hai

- **Training** = weights seekhna (ek baar hota hai, mahino lagte hain).
- **Inference** = trained model ko use karke naya text generate karna (jab tum ChatGPT pe type karte ho, ye ho raha hai).

Karpathy: *"Inference is just predicting from these distributions one at a time."*

- Model har step pe **next token ki probability distribution** nikalta hai (e.g., "the" 40%, "a" 20%, ...).

- Ek token **randomly (stochastically)** us distribution se sample hota hai — **isi liye same prompt se har baar thoda different output aa sakta hai.**
- Ye chosen token wapas input mein add hota hai, aur process repeat — ek-ek token karke poora response ban jaata hai.

Important clarity (Karpathy): *“When you’re on ChatGPT... that model is trained... by OpenAI many months ago.”* — Matlab jab tum chat karte ho, koi naya “learning” nahi ho raha real-time mein — model already-trained hai, sirf inference chal raha hai. Har conversation se woh “seekhta” nahi (unless company explicitly fine-tune kare future mein).

3. Base Model ka Real Behavior — Hallucination aur In-Context Learning

3.1 Hallucination kya hai

- Base model sirf **probabilistically next best guess** deta hai — chahe fact sahi ho ya galat, confidence same tone mein bolega.
- Isko **hallucination** kehte hain — jab model fact bana ke (fabricate) bolta hai, but confidently.

Karpathy: *“All of what we’re seeing here is what’s called hallucination. The model is just taking its best guess in a probabilistic manner.”*

3.2 Few-shot Prompting (Base model ko useful banane ka trick)

- Base model ko bina fine-tuning ke bhi useful bana sakte ho — sirf prompt mein examples de kar.
- Isse **in-context learning** kehte hain — model prompt ke andar diye gaye pattern ko dekh kar copy karta hai.

Example:

English: hello → French: bonjour
 English: goodbye → French: au revoir
 English: thanks → French:

Model pattern dekh ke “merci” complete kar dega — bina explicitly train hue.

4. STAGE 2 — Post-Training: Supervised Fine-Tuning (SFT) → Assistant Banana

- Base model sirf text-continuer hai. Isko **helpful assistant** banane ke liye **post-training** hoti hai.
- Yahan **human labelers** (professional annotators) **ideal conversations** likhte hain — question + best possible assistant response.

Karpathy: *“There are human labelers involved whose job it is professionally to create these conversations... and they give the ideal assistant response.”*

- Model in conversations ko **imitate** karna seekhta hai — jaise ek naya employee jo senior ke sample answers dekh kar apna style dhaalta hai.
- Goal: model **helpful, truthful, harmless** persona adopt kare.

4.1 Conversation Tokenization — Special Tokens

- Conversations ko bhi tokens mein encode karna padta hai — for this, special tokens use hote hain jo batate hain:
 - Turn kab start/end hua (<|user|>, <|assistant|> jaise markers)
 - Kaun bol raha hai (role: system/user/assistant)

Karpathy: *“We need some kind of data structures... rules around how conversations get encoded and decoded to and from tokens.”*

Important insight (post-training vs pre-training): - Pre-training: **bahut compute-heavy** (weeks/months, trillions of tokens). - Post-training (SFT): **comparatively halka** hota hai — kam data, kam time — kyunki base model already language/knowledge jaanta hai, sirf **format aur behavior** sikhaya jaa raha hai.

5. LLM Psychology — Model ke “Dimaag” ko Samajhna

Ye section sabse important hai practically use karne ke liye.

- **Parameters (weights) mein stored knowledge** = ek **vague, fuzzy recollection** jaisa hai (jaise tumhe kisi purani kitab ka mota-mota yaad ho, exact quote nahi).
- **Context window (jo tokens tum prompt mein daalte ho)** = model ki **working memory** — jo abhi “saamne” hai, uspe model bahut accurate/reliable hai.

Karpathy: *“Knowledge in the parameters of the neural network is a vague recollection... the knowledge in the tokens that make up the context window is the working memory.”*

Practical takeaway (bahut zaroori): - Agar koi specific fact, document, ya data chahiye — usko **context mein paste karo** (copy-paste karke do), sirf “model ko yaad hoga” pe depend mat karo. - Isi principle pe **RAG (Retrieval-Augmented Generation)** based tools kaam karte hain — pehle relevant data dhoondo, phir context mein daal ke poocho.

6. Limitations — Model Kyun Kuch Cheezon Mein Weak Hai

6.1 Counting & Spelling problem

- Model **characters nahi dekhta — sirf tokens dekhta hai**.
- Example: “strawberry” word ek single token ho sakta hai model ke liye — usko pata hi nahi ki isme kitne “r” hain, kyunki woh letter-by-letter nahi soch raha.

Karpathy: *“The models don’t see the characters; they see tokens, and their entire world is about tokens.”*

6.2 Per-token computation limit

- Model **left-to-right, ek dimension mein** kaam karta hai, aur **har single token pe limited/fixed computation** hoti hai.
- Agar tum ek complex math problem ka answer **directly ek hi token mein maango** (jaise “seedha final answer bolo”), model galat guess kar sakta hai — kyunki utna complex calculation ek token ke andar possible hi nahi.

Karpathy: *"If you are answering the question directly and immediately, you are training the model to try to basically guess the answer in a single token, and that is just not going to work."*

Solution / Best Practice: - Model ko **step-by-step (chain of thought) soch ke likhne do** — isse computation multiple tokens mein distribute ho jaata hai, accuracy badh jaati hai. - Complex arithmetic/counting ke liye, model ko **tool use karne do** (jaise code interpreter/calculator) — khud "mental math" pe depend mat karo.

Karpathy: *"You may want to basically just ask the model to use the code interpreter."*

Mera addition: Isi wajah se prompting mein "Let's think step by step" jaisi techniques kaam karti hain — ye model ko tokens ke through reasoning "spread" karne ka mauka deti hai.

7. Mitigating Hallucinations (Model ko Zyada Truthful Banana)

Do main techniques:

1. Training data mein "I don't know" examples daalna

- Company empirically test karti hai ki model ko kya pata hai, kya nahi.
- Jahan model confidently galat bolta hai, wahan training set mein example daala jaata hai jisme correct response ho: *"I don't know"* ya *"I'm not sure"*. > Karpathy: *"We can empirically probe the model to figure out what it knows and doesn't know... and then add examples to the training set."*

2. Tool use (web search)

- Model ko sikhaaya jaata hai ki jab uncertain ho ya recent/updated info chahiye ho, to **web search tool call kare** — apne "purane vague memory" pe depend karne ke bajaye.
- Isse factual accuracy badh jaati hai kyunki fresh, verifiable data mil jaata hai.

Mera addition: Isi liye jab bhi koi current event, latest price, ya specific number poochho, achha LLM product (jaise Claude, ChatGPT) automatically web search karta hai instead of guessing from memory.

8. STAGE 3 — Reinforcement Learning (RL): Reasoning Sikhaana

Ye teesra aur sabse advanced stage hai — abhi ke "thinking models" (o1, o3, DeepSeek-R1, Gemini Thinking) yahi use karte hain.

8.1 RL kya hai

- Model ko ek problem diya jaata hai, aur usse **khud multiple solutions try karne** diye jaate hain (trial and error).
- Jo solution sahi answer tak pahunchta hai, uske "token path" ko **reinforce** (reward/strengthen) kiya jaata hai — taaki future mein model us jaisi strategies zyada use kare.

Karpathy: *"Reinforcement learning is still kind of thought to be under the umbrella of post-training, but it is the last third major stage."*

8.2 Verifiable Domains

- RL sabse achha kaam karta hai jahan answer **objectively check ho sake** — jaise **Math** (answer sahi/galat) aur **Code** (test cases pass/fail).
- Model in domains mein practice karke apna **emergent chain-of-thought reasoning** develop karta hai — ye explicitly kisi ne sikha nahi, ye **optimization ka side-effect** hai.

Karpathy: *“The model learns what we call these chains of thought in your head, and it’s an emergent property of the optimization.”*

8.3 AlphaGo Analogy (bahut powerful example)

- **AlphaGo** (Go game khelne wala AI) ne RL use karke insaano se better khela — sirf human games imitate nahi kiye, balki **khud khel khel ke naye strategies discover kiye**.
- Famous: **“Move 37”** — ek aisi move jo koi human expert kabhi nahi khelta, but statistically woh jeetne mein help karti thi. Baad mein experts ne realize kiya ki ye actually brilliant move thi.

Karpathy: *“AlphaGo in the process of reinforcement learning discovered kind of like a strategy of playing that was unknown to humans... but is in retrospect brilliant.”*

Takeaway: RL sirf imitation se aage jaake, LLM ko **naye tareeke khud dhoondne** deta hai jo insaano ne kabhi socha bhi nahi — isi wajah se reasoning models kabhi-kabhi “weird lekin effective” reasoning steps generate karte hain.

8.4 Unverifiable Domains ka Challenge — RLHF

- Math/Code jaise “clean” answer wale domains ke alawa, creative writing, jokes, summaries jaise tasks mein “sahi/galat” ka koi fixed check nahi hai — ye **subjective** hai.
- Solution: **RLHF (Reinforcement Learning from Human Feedback)**
 - Ek alag neural network banayi jaati hai jise **Reward Model** kehte hain — ye human judgment ko **imitate/simulate** karta hai (human ratings pe train hoke).
 - Phir LLM is reward model ke against RL train hota hai.

Karpathy: *“We basically train a whole separate neural network that we call a reward model and this neural network will kind of like imitate human scores.”*

8.5 RLHF ki Limitation

- Reward model ek **“lossy simulation”** hai real human judgment ka — perfect nahi hai.
- RL bahut powerful hone ki wajah se, model **reward model ko “game” karna** seekh sakta hai — matlab high score paane ke liye aise outputs bana sakta hai jo actually non-sensical/meaningless hain but reward model ko “achhe” lagte hain (adversarial exploitation).

Karpathy: *“Reinforcement learning is extremely good at discovering a way to game the model to game the simulation.”*

Mera addition: Isi wajah se RLHF ko limited steps tak hi run kiya jaata hai (early stopping) — zyada der RL chalane se model reward hacking start kar deta hai. Ye ek active research problem hai.

9. Quick Comparison — GPT-2 vs Llama 3 (Scale ka Evolution)

	GPT-2 (2019)	Llama 3 (2024)
Parameters	~1.6 billion	~45 billion (per Karpathy's example, bigger variants exist)
Training tokens	~100 billion	~15 trillion
Significance	Pehli baar "modern LLM stack" (transformer + next-token training) ek jagah aaya	Scale, quality, aur reasoning mein bahut aage — dono base + instruct versions available

Karpathy: *"GPT-2 was the first time that a recognizably modern stack came together."*
"Llama 3 is a much bigger model and much more modern model."

Mera addition: Ye table dikhata hai ki pichle 5 saal mein sirf "scale" (data + parameters + compute) badhne se hi models kitna better ho gaye — isko **"Scaling Laws"** kehte hain, jo AI progress ka ek core driving factor raha hai.

10. Thinking Models — DeepSeek, o1/o3, Gemini Thinking

- Ye models RL-heavy training se banaye gaye hain — inference ke time zyada "sochne" (extra reasoning tokens generate karne) ke liye designed hain.
- Karpathy ke mutabik, DeepSeek jaise models current top models ke **on par** hain, though evaluation tricky hai.
- **DeepSeek** ek **open-weights, MIT-licensed** model hai — koi bhi download/use/modify kar sakta hai (bade proprietary labs ke against ek open alternative).

Karpathy: *"Deep Seek is an MIT license model, it's open weights anyone can use these weights."*

11. Practical Guide — LLM ko Effectively Use Kaise Karein

Karpathy ke tips + mere additions:

1. **Model ko ek tool samjho, oracle nahi.** Har output verify karo — especially facts, numbers, dates. > *"Treat this as what it is, which is a stochastic system that is really magical but that you can't also fully trust."*
2. **Important context khud provide karo** — "working memory" (context window) pe rely karo, "vague memory" (parameters) pe nahi.
3. **Complex reasoning tasks mein "step-by-step socho" bolo** — direct answer maangne se galti ki chance badh jaati hai.
4. **Counting/math/precise calculation ke liye tool use karne do** (code interpreter, calculator) — mental arithmetic pe bharosa mat karo.
5. **Multiple models compare karo** jab critical decision ho — ek model ka answer final truth mat maano.
6. **Latest info ke liye web-search-enabled mode use karo** — training-cutoff wale purane facts pe depend mat karo.

Mera extra addition (practical prompting techniques): - **Be specific** — clear instructions, format specify karo (e.g. “answer in bullet points, under 100 words”). - **Give examples (few-shot)** — agar specific style/format chahiye to 1-2 example do. - **Break big tasks into steps** — ek hi mega-prompt ki jagah task ko chunks mein do. - **Ask it to cite/verify** — critical use-cases mein sources maango.

12. Future of LLMs (Karpathy ki Predictions)

1. **Multimodality** — models text, image, audio sab ek saath seamlessly handle karenge (already ho raha hai — GPT-4o, Gemini, Claude).
2. **Long-running Agents** — models jo ek task ko **hours/days tak khud manage** karenin, multiple steps mein, bina baar-baar human input ke.
3. **Invisible Integration** — AI everyday tools (email, browser, docs, code editors) mein itna ghul-mil jayega ki alag se “AI use kar raha hoon” jaisa feel nahi hoga — bas kaam ho jayega.

Karpathy: “The models will very rapidly become multimodal.” “We’re going to start to see what’s called agents which perform tasks over time.”

13. Staying Updated (Resources jo Karpathy suggest karte hain)

- **LM Arena (El Marena)** — models ki live ranking/comparison.
 - **AI News** newsletters.
 - **X (Twitter)** — researchers/labs real-time updates dete hain.
 - **Open-weight models** (jaise DeepSeek) — khud experiment karne ke liye download/run kar sakte ho.
-

14. One-Page Cheat Sheet (Revision ke liye)

INTERNET TEXT → [Filter/Clean] → TOKENIZE (BPE) → TRAIN NEURAL NET
(predict next token)
↓
BASE MODEL
(hallucinate, in-context learn)
↓
SFT: human-written ideal conversations → ASSISTANT MODEL
↓
RL: trial & error on math/code (verifiable) + RLHF (unverifiable)
↓
REASONING / THINKING MODEL

3 Golden Rules to Remember: 1. Params = fuzzy memory | Context = sharp working memory → important info context mein do. 2. Ek token mein limited computation → complex answers ko steps mein todo. 3. Model stochastic hai → verify everything, especially facts/numbers.

Glossary (Key Terms — Hinglish mein)

Term	Matlab
Token	Text ka chhota piece jisse model process karta hai (word/sub-word/character group)
Parameter/Weight	Network ke andar ke numbers jo training se seekhe jaate hain
Pre-training	Internet text pe next-token prediction training (base knowledge)
Base Model	Sirf text-continuer, assistant nahi
SFT (Supervised Fine-Tuning)	Human conversations pe train karke assistant banana
RL (Reinforcement Learning)	Trial-error se best solution paths reinforce karna
RLHF	Human feedback ko simulate karne wale reward model ke against RL
Hallucination	Model ka confidently galat fact bolna
In-context learning	Prompt ke andar diye examples se pattern seekhna (bina retraining)
Context window	Wo tokens jo abhi model ke “saamne” hain (working memory)
Reward hacking	Model ka reward model ko “game” karke fake-high score paana
Chain of Thought	Step-by-step reasoning jo model generate karta hai answer se pehle

Notes tayyar hain — agar kisi specific section (jaise Transformer architecture, RLHF math, ya tokenization BPE algorithm) ko aur deep dive karna ho, bata dena, uska alag detailed note bana dunga.